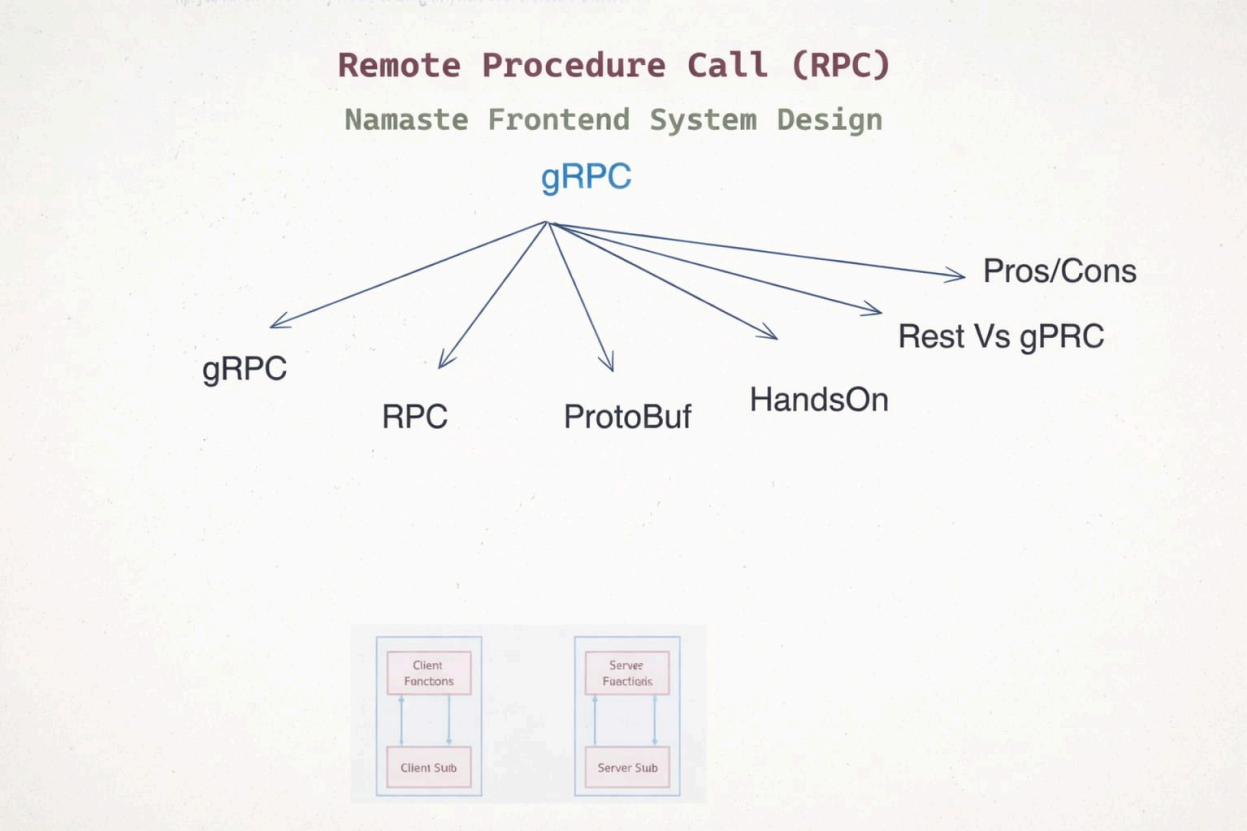
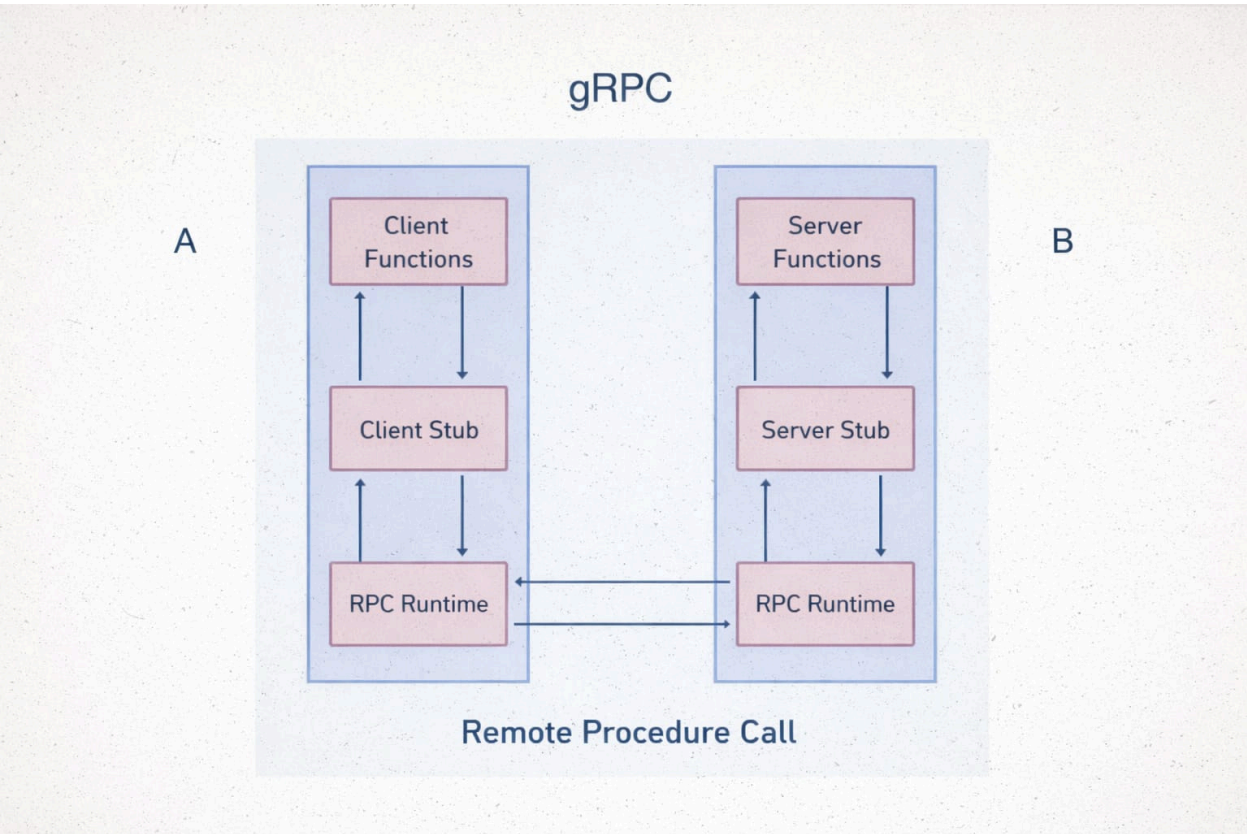
gRPC


Remote Procedure Call (RPC)
Namaste Frontend System Design
gRPC
gRPC
RPC
ProtoBuf
HandsOn
Rest Vs gPRC
Pros/Cons
Client Fanctons
Client Stub
Server Fuactions
Server Stub



What is gRPC

gRPC (Google Remote Procedure Call) is a framework that lets a client call functions on a remote server as if they were local functions.

Why use gRPC

- Faster than REST (binary instead of JSON)
- Strongly typed (uses `.proto` contracts)
- Built for service-to-service communication

How it works (based on your diagram)

1. **Client Function**
    - You call a function like `getUser(id)`
2. **Client Stub**

- Acts like a proxy
- Converts function call → Protocol Buffer (binary format)

3. **RPC Runtime**
   - Sends request over network using HTTP/2
4. **Server Stub**
   - Converts binary data back to usable format
   - Calls server function
5. **Server Function**
   - Executes logic and returns response
6. **Response path**
   - Server → Server Stub → RPC Runtime → Client Stub → Client

Key concept

- **Client Stub = like GraphQL resolver + API call layer**
- **RPC Runtime = network layer**
- **ProtoBuf = schema + data format**

Protocol Buffers (ProtoBuf)

Protocol Buffers is a **data format + contract system** used by gRPC.

It defines how data is structured and transferred between client and server.

Interface Definition Language (IDL)

ProtoBuf uses `.proto` files as an **IDL**.

Example:

```
syntax = "proto3";

service UserService {
```

```
  rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
  int32 id = 1;
}

message UserResponse {
  string name = 1;
}
```

What this does:

- Defines **API methods** (`rpc GetUser`)
- Defines **data structure** (`UserRequest`, `UserResponse`)
- Acts as a **contract** between client and server

Serialization / Deserialization

Serialization

- Convert structured data → binary format
- Happens at **Client Stub**

```
{ id: 101 } → binary
```

Deserialization

- Convert binary → structured data
- Happens at **Server Stub**

```
binary → { id: 101 }
```

Binary Support (Why it matters)

ProtoBuf uses **binary encoding instead of JSON**

Benefits:

- Smaller size
- Faster transfer
- Less CPU usage
- Strong typing (no parsing ambiguity)

Key Flow

```
JS Object
 → Serialize (ProtoBuf)
 → Binary (over network)
 → Deserialize
 → Server Object
```

Benefits of gRPC

- **High performance**

  Uses binary (ProtoBuf) instead of JSON → smaller payloads, faster communication

- **Strong typing**

  `.proto` contracts ensure strict request/response structure → fewer runtime errors

- **Code generation**

  Automatically generates client + server code from `.proto` → less manual work

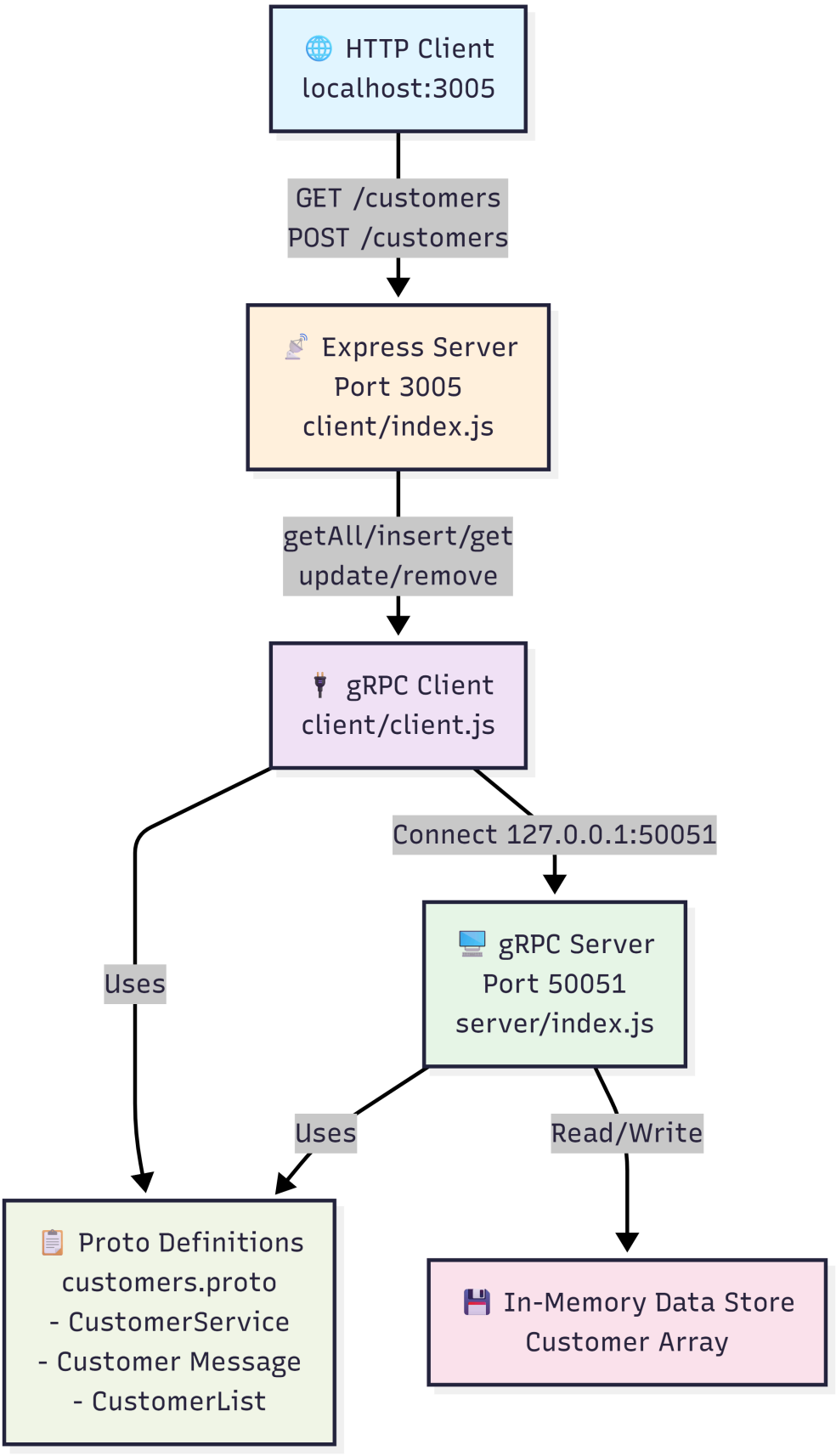
- **HTTP/2 support**
  - Multiplexing (multiple requests in one connection)
  - Header compression
  - Faster than HTTP/1.1

- **Streaming support**
  - Client streaming
  - Server streaming
  - Bidirectional streaming
- **Cross-language support**

  Works across multiple languages (Node.js, Java, Go, Python, etc.)
- **Better for microservices**

  Efficient internal communication between services

Code and Architecture


HTTP Client
localhost:3005
GET /customers
POST /customers
Express Server
Port 3005
client/index.js
getAll/insert/get
update/remove
gRPC Client
client/client.js
Connect 127.0.0.1:50051
Uses
gRPC Server
Port 50051
server/index.js
Uses
Read/Write
Proto Definitions
customers.proto
- CustomerService
- Customer Message
- CustomerList
In-Memory Data Store
Customer Array

1. Initialize gRPC Server (setup)

```
npm init -y
npm install @grpc/grpc-js @grpc/proto-loader
```

2. customers.proto

```
syntax = "proto3";

message Empty {}

message CustomerRequestId {
    string id = 1;
}

message Customer {
    string id = 1;
    string name = 2;
    int32 age = 3;
    string address = 4;
}

message CustomerList {
    repeated Customer customer = 1;
}

service CustomerService {
    rpc GetAll (Empty) returns (CustomerList);
    rpc Get (CustomerRequestId) returns (Customer);
    rpc Insert (Customer) returns (Customer);
    rpc Update (Customer) returns (Customer);
    rpc Remove (CustomerRequestId) returns (Empty);
}
```

3. server/index.js

```
import { fileURLToPath } from "url";
import { dirname, join } from "path";

const __dirname = dirname(fileURLToPath(import.meta.url));
const PROTO_PATH = join(__dirname, "../customers.proto");

import grpc from "@grpc/grpc-js";
import protoLoader from "@grpc/proto-loader";

const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  keepCase: true,
  longs: String,
  enums: String,
  arrays: true,
});

const customersProto = grpc.loadPackageDefinition(packageDefi
nition);

const server = new grpc.Server();

const customers = [
  {
    id: "sfdjkfjsdf",
    name: "John Doe",
    age: 30,
    address: "123 Main St",
  },
  {
    id: "fw2342df",
    name: "Jane Smith",
    age: 25,
    address: "456 Oak Ave",
```

```
  },
  {
    id: "sdf2342",
    name: "Bob Johnson",
    age: 40,
    address: "789 Pine Rd",
  }
];

server.addService(customersProto.CustomerService.service, {
  getAll: (call, callback) => {
    callback(null, { customer: customers });
  },
  get: (call, callback) => {
    const customer = customers.find((c) => c.id === call.requ
est.id);
    if (customer) {
      callback(null, customer);
    } else {
      callback({
        code: grpc.status.NOT_FOUND,
        details: "Customer not found",
      });
    }
  },
  insert: (call, callback) => {
    const newCustomer = {
      id: Math.random().toString(36).substring(2, 15),
      name: call.request.name,
        age: call.request.age,
        address: call.request.address,
    };
    customers.push(newCustomer);
    callback(null, newCustomer);
  },
```

```
  update: (call, callback) => {
    const index = customers.findIndex((c) => c.id === call.re
quest.id);
    if (index !== -1) {
      customers[index] = {
        id: call.request.id,
        name: call.request.name,
        age: call.request.age,
        address: call.request.address,
      };
      callback(null, customers[index]);
    } else {
      callback({
        code: grpc.status.NOT_FOUND,
        details: "Customer not found",
        });
    }
  },
  remove: (call, callback) => {
    const index = customers.findIndex((c) => c.id === call.re
quest.id);
    if (index !== -1) {
      const removedCustomer = customers.splice(index, 1)[0];
      callback(null, removedCustomer);
    } else {
      callback({
        code: grpc.status.NOT_FOUND,
        details: "Customer not found",
      });
    }
  },
});

server.bindAsync("127.0.0.1:50051", grpc.ServerCredentials.cr
eateInsecure(), (err, port) => {
  if (err) {
```

```
    console.error("Failed to bind server:", err);
  } else {
    console.log(`Server bound to port ${port}`);
    server.start();
  }
});
```

4. client/client.js

```
import { fileURLToPath } from "url";
import { dirname, join } from "path";

const __dirname = dirname(fileURLToPath(import.meta.url));
const PROTO_PATH = join(__dirname, "../customers.proto");

import grpc from "@grpc/grpc-js";
import protoLoader from "@grpc/proto-loader";

const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  keepCase: true,
  longs: String,
  enums: String,
  arrays: true,
});

const CustomerService = grpc.loadPackageDefinition(packageDef
inition);

export const client = new CustomerService.CustomerService(
  "127.0.0.1:50051",
  grpc.credentials.createInsecure()
);
```

5. client/index.js

```
import express from "express";
import {client} from "./client.js";

const app = express();
const PORT = 3005;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.get('/',(_req,res)=>res.json({message:"Root is groot"}))

app.get('/customers',(req,res)=>{
    client.getAll({},(err,response)=>{
        if(err){
            console.error(err);
            res.status(500).json({error:"Internal Server Erro
r"});
        }else{
            res.json(response);
        }
    })
}
)

app.post('/customers',(req,res)=>{
    const {id,name,age,address} = req.body;
    client.insert({id,name,age,address},(err,response)=>{
        if(err){
            console.error(err);
            res.status(500).json({error:"Internal Server Erro
r"});
        }
        else{
```

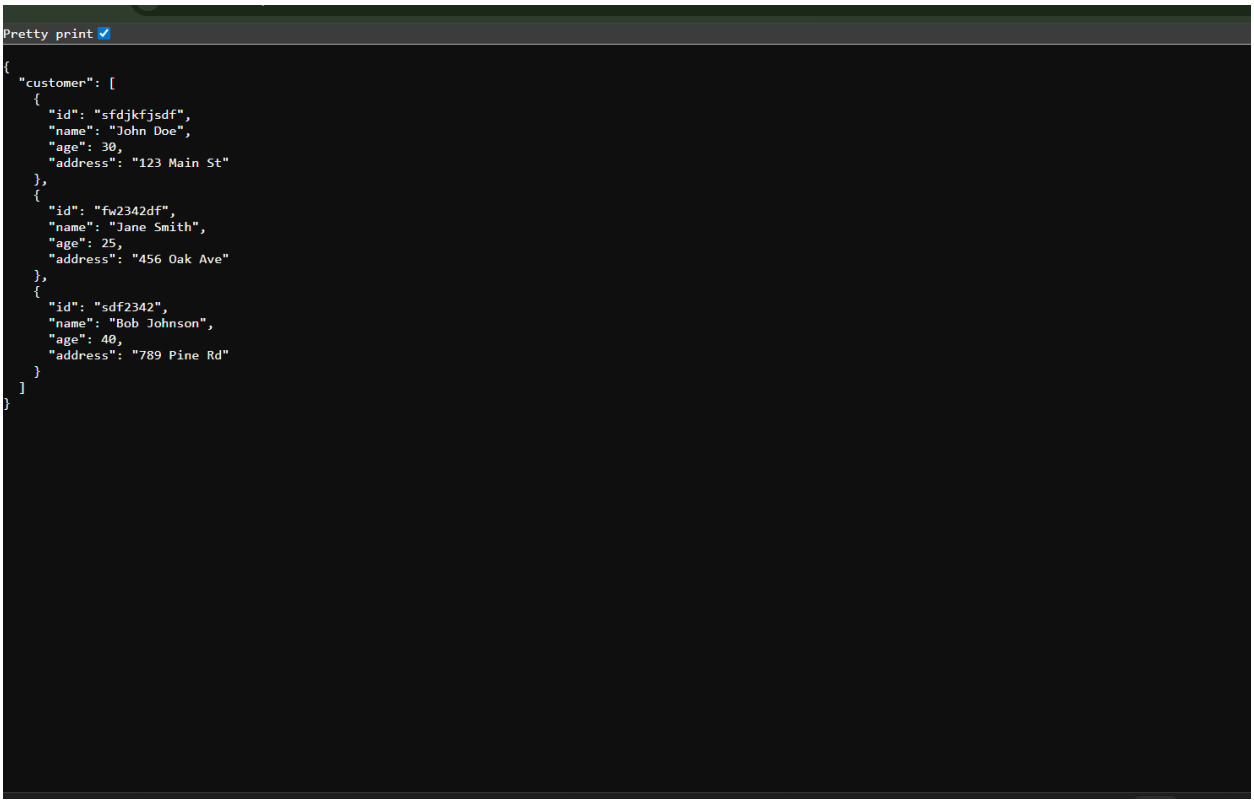
```
            res.json(response);
        }
    })
})

//more routes can be added here similarly for update and dele
te operations


app.listen(PORT,()=>console.log(`Client Started at port ${POR
T}`))
```

Flow

```
client/index.js → client.js → gRPC call → server/index.js → r
esponse
```



REST vs gRPC

Feature	REST	gRPC
Communication Protocol	HTTP/1.1 or HTTP/HTTPS	HTTP/2 (default)
Message Format	JSON / XML (text-based)	Protocol Buffers (binary)
IDL (Interface Definition Language)	No strict standard (OpenAPI / Swagger / WSDL)	`.proto` files (strict contract)
Data Serialization	Text-based (JSON, XML)	Binary (ProtoBuf)
Efficiency	Slower (larger payloads, multiple requests)	Faster (compact + multiplexing in HTTP/2)
Flexibility	More flexible (loose schema)	Strict typing (strong contract)
RPC Styles	Request–Response (GET, POST, etc.)	Unary, Client Streaming, Server Streaming, Bidirectional
Streaming	Limited (WebSockets / SSE workaround)	Native streaming support
Performance	Moderate	High performance
Use Case	Public APIs, web apps	Microservices, real-time systems
Learning Curve	Easy	Slightly steep

gRPC: Advantages vs Disadvantages

Advantages	Disadvantages
High performance (binary + HTTP/2)	Not browser-friendly (needs gRPC-Web)
Smaller payload size (ProtoBuf)	Harder to debug (binary data)
Strong typing with `.proto` contracts	Learning curve (IDL + tooling)
Supports streaming (bi-directional)	Limited human readability
Efficient for microservices communication	Less flexible than REST
Built-in code generation	Requires schema management